

Highlights and Hidden Gems in SQL Server 2025

Matt Gordon

He/him

Director of Data and Analytics

Lean TECHniques

Matt Gordon

He/him

Director of Data & Analytics
Lean TECHniques

mgordon@leantechniques.com

@sqlatspeed everywhere online



Who Is Here and What Are You Hoping to Learn?

- DBAs? DBAs who are AG Geeks?
- Database Developers? Data Engineers? Developers?
- What SQL Versions Are We On?
- New Stuff?
- Improved Stuff?
- Stuff That Isn't Great?

Building on a foundation of innovation



SQL Server 2017

SQL Server on Linux
Containers
Adaptive query processing
Automatic Tuning
Graph database
Machine learning services



SQL Server 2019

Data virtualization
Intelligent query processing
Accelerated database recovery
Data classification



SQL Server 2022

Cloud Connected
IQP NextGen
Ledger
Data Lakes
T-SQL enhancements

Introducing SQL Server 2025



"The AI-ready enterprise database from ground to cloud"



Five pillars: AI Built-In, Best-in-class Security & Performance, Made for Developers, Cloud Agility through Azure, Fabric Integration



aka.ms/sqlserver2025

SQL Server 2025 – Areas of Innovation

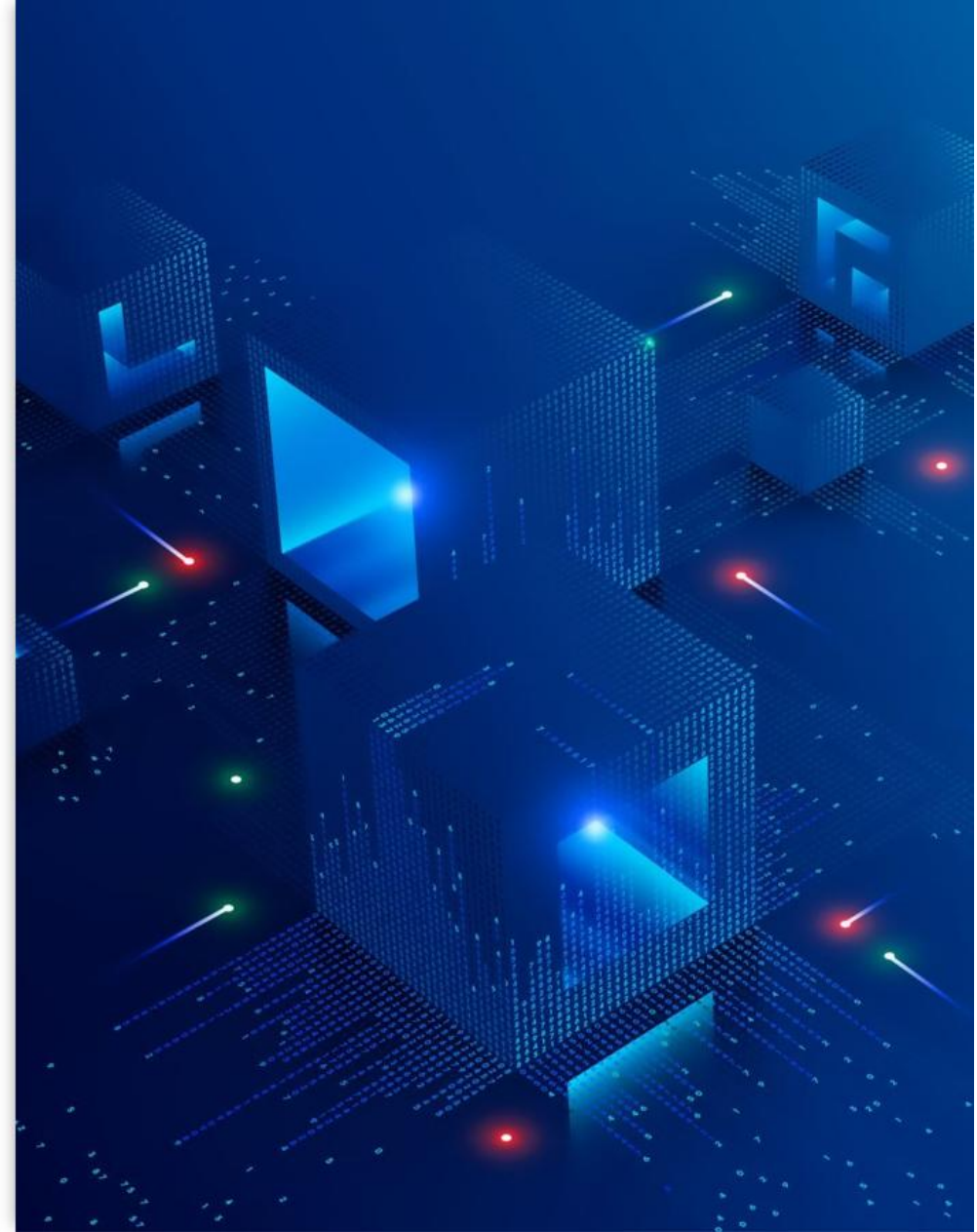
AI Built-In: Vectors, embeddings,
model management, RAG patterns

**Develop Modern Data
Applications:** JSON, RegEx, REST
API, CES, GitHub Copilot for SQL

Integrate Your Data with Fabric:
Mirroring, near-real-time analytics

The SQL Server 2025 Platform Architecture

- Outbound connections: Microsoft Foundry, Event Hub, Fabric, Entra, any other REST API you want
- Local connections: AI models (Ollama, ONNX), REST endpoints, Semantic Kernel
- Management plane: Arc, SSMS 22 with Copilot, VS Code mssql extension
- Secondary replica capabilities: Query Store, backups, tuning, diagnostics

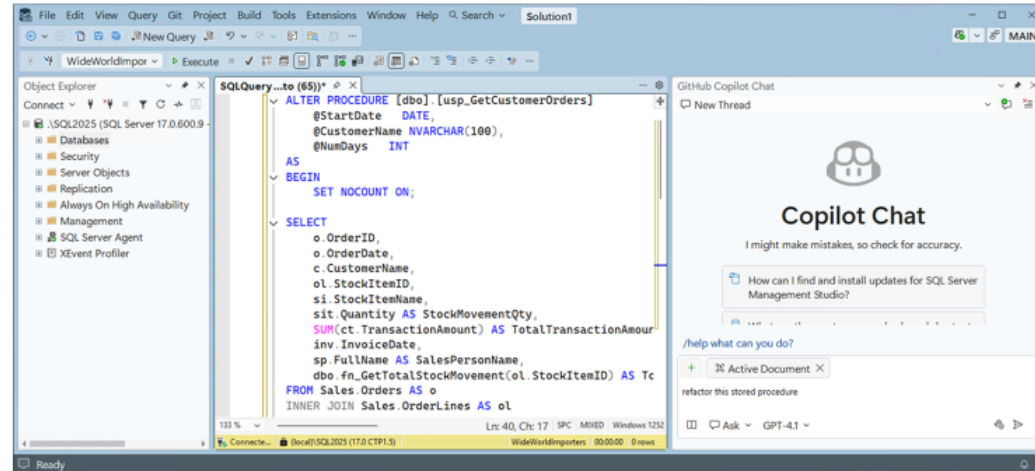


SQL Server 2025

Developer Experiences

SSMS 22 – General Availability

- Based on Visual Studio 2022 (64-bit)
- Copilot integration (public preview)
- Dark theme and multiple color themes
- Git and TFS support
- New connection experience
- Windows ARM64 support



SSMS 22 – Developer Productivity Features

Query editor
improvements

JSON viewer

Query hints
recommendation

Zooming for grid
results

Always Encrypted
assessment

VS Installer /
update
experience

VS Code mssql Extension

RIP Azure Data Studio
2017-2026

GitHub Copilot integration for SQL authoring

Cross-platform SQL development (Windows, Mac, Linux)

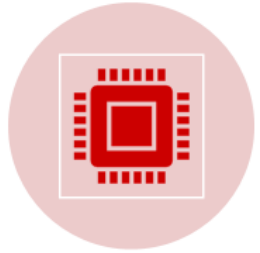
Schema-aware IntelliSense

Modern development workflow integration

*Query Store Extension

<https://github.com/jdanton/query-store-reports>

Built for Developers – Overview



This is the most developer-friendly SQL Server release in a decade



Native JSON, RegEx, REST APIs, Change Event Streaming



GitHub Copilot for SQL, Python driver, Data API Builder



GraphQL via Data API Builder

Microsoft Python Driver for SQL Server



Official Microsoft Python driver



First-class Python support for SQL
Server connectivity



Integrates with popular data science
and ML frameworks



Complements the existing ODBC, JDBC,
and .NET drivers

SQL Server 2025

Data Storage and T-SQL Enhancements

Native JSON – The Problem



Developers have been storing JSON in varchar/nvarchar columns for years



No validation, no specialized indexing, no binary optimization

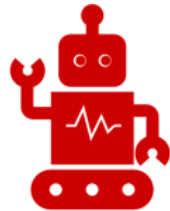


JSON functions worked, but storage was inefficient

Native JSON – The Solution



New JSON data type:
binary storage, up to 2GB



Automatic validation on
insert/update



Compressed storage: less
I/O, fewer page reads

Native JSON - Indexing

Dedicated JSON
index type

Optimized for JSON
path queries

Replaces the
workaround of
computed columns
+ standard indexes

Native JSON – New Functions

JSON_OBJECTAGG — aggregate rows into a JSON object

JSON_ARRAYAGG — aggregate rows into a JSON array

JSON_CONTAINS — test for membership

JSON_QUERY WITH ARRAY WRAPPER — wrap results as array

Native JSON

— Use Cases

REST APIs and microservice payloads

Configuration storage

Document-style flexible schemas

E-commerce product catalogs with variable attributes

Log data and telemetry

T-SQL Enhancements — Regular Expressions

REGEXP_LIKE — test if a string matches a pattern

REGEXP_COUNT — count pattern occurrences

REGEXP_INSTR — find position of pattern match

REGEXP_REPLACE — replace pattern matches

REGEXP_SUBSTR — extract substring matching pattern

No more CLR functions or LIKE workarounds for pattern matching

T-SQL Enhancements — String and Data Functions

`||` string concat operator (ANSI standard)

`CURRENT_DATE` returns DATE type

`SUBSTRING()` with optional length — omit to get "rest of string"

`UNISTR()` — Unicode string construction

`DATEADD()` now supports bigint

T-SQL Enhancements — Aggregates and Matching

PRODUCT() aggregate
— multiplicative
aggregate function

Fuzzy string match
functions —
approximate matching
for data quality

BASE64_ENCODE /
BASE64_DECODE —
binary-to-text and
back

GitHub Copilot for SQL – Overview



AI-assisted T-SQL authoring in VS Code and SSMS 22



Context-aware: understands your schema, tables, and relationships



Generate queries from natural language descriptions



Explain existing complex queries



Suggest optimizations

GitHub Copilot for SQL – In Action



Inline suggestions as you type



Chat-based query generation: "Show me top 10 customers by revenue last quarter"



Query explanation: highlight a query, ask "What does this do?"



Schema exploration: "What tables relate to orders?"

SQL Server 2025

Event-driven Architecture

Support

Change Event Streaming (CES) – The Problem



Change Data Capture (CDC) and Change Tracking (CT) are pull-based — applications must poll for changes



Modern architectures demand push-based, event-driven patterns



Microservices, real-time analytics, and AI agents need instant notification

Change Event Streaming (CES) — The Solution



Stream data changes in near-real-time from SQL directly to **Azure Event Hubs (or Fabric Eventstreams)**



Push-based delivery



"IO-less" — reads committed transactions from the transaction log



Supports DDL changes



Single destination per configuration

CT vs. CDC vs. CES — Comparison

Change Tracking (CT): Sync capture, system tables (minimal), row identifier only, pull-based, immediate availability

Change Data Capture (CDC): Async from log, system tables (extra logging), row identifier + data, pull-based, near-real-time, requires SQL Agent

Change Event Streaming (CES): Async from log, event stream ("IO-less"), row identifier + data + DDL, push-based, near-real-time

CT vs. CDC vs. CES – When to Use What

CT: Cache invalidation, sync scenarios

CDC: Data warehousing, auditing, ETL pipelines

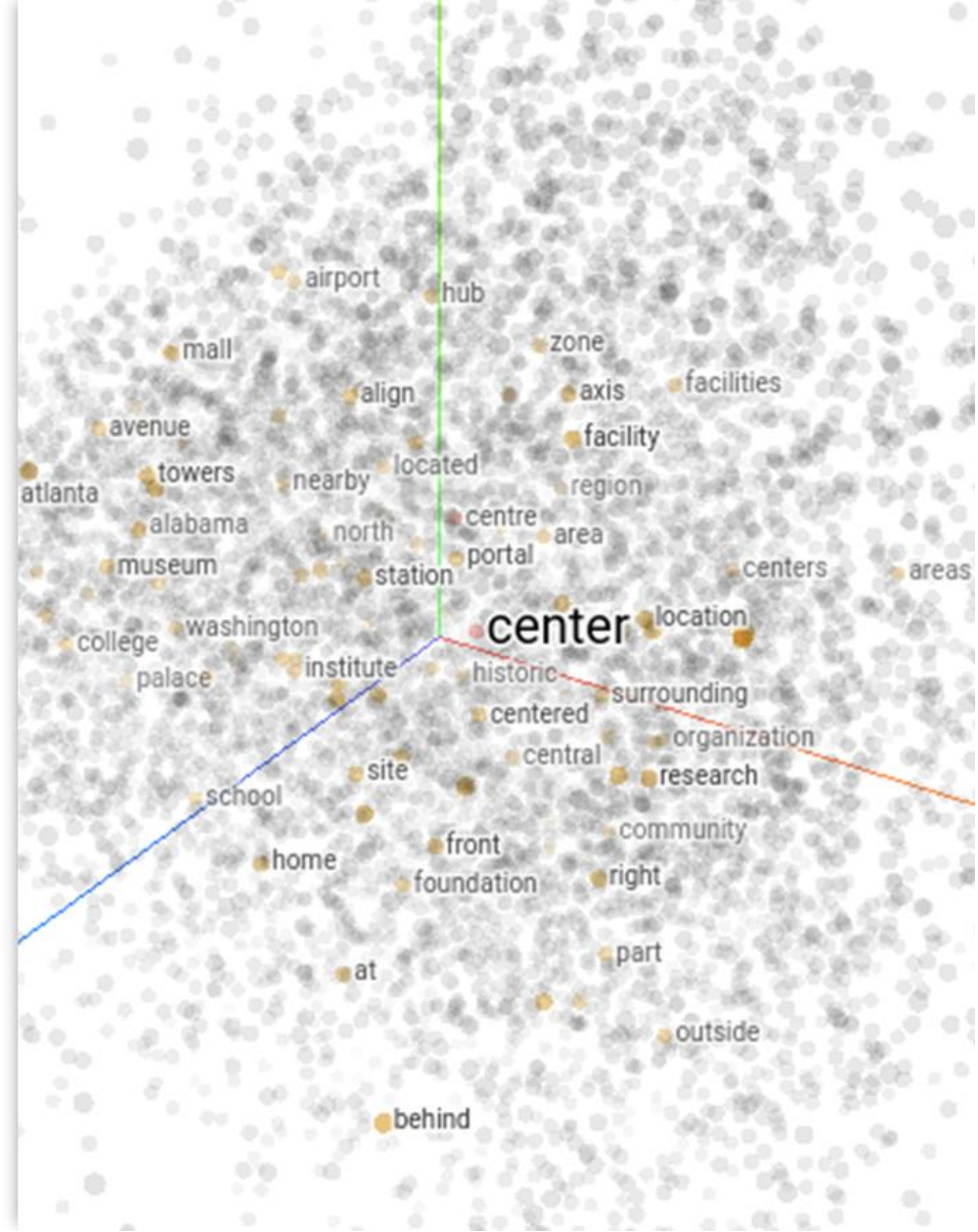
CES: Event-driven architectures, microservices integration, real-time analytics, cache sync, AI Agents

SQL Server 2025

AI-related Support

Vector Search – The Concept

- Traditional search: keyword matching (LIKE, Full-Text Search)
- Semantic search: meaning-based matching using embeddings
- Embeddings: numerical representations that capture meaning
- "The prompt may contain words not found in your data!" — this is why vector search matters



Vector Search with Your Data – The RAG Pattern

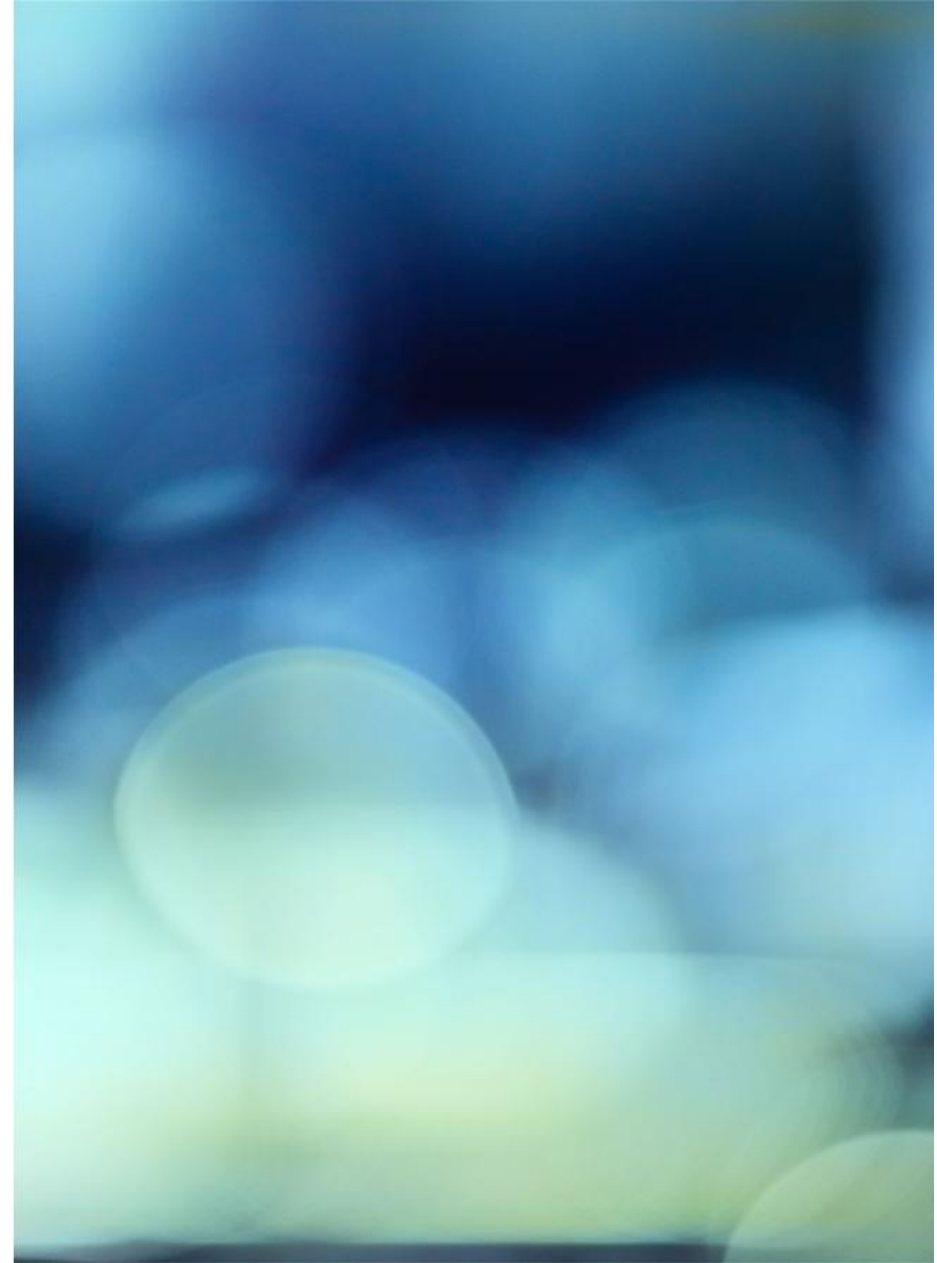
Prompt → Embedding Model → Vector embedding

Vector embedding compared to stored embeddings via similarity search

Results augment the original prompt

Augmented prompt sent to Language Model → Generated response

All under SQL Server security



Agentic RAG – Three Patterns

- **Vector Search:** Retrieve most semantically relevant data to ground LLMs
- **Operational RAG:** Store vectors and data together for consistency
- **Structured Queries:** Allow LLMs to query structured data with rich metadata and query optimization

Good repo for testing: <https://github.com/Azure-Samples/azure-sql-db-session-recommender>

The Vector Data Type

New native vector data type

Stores arrays of floating-point numbers

Configurable dimensions (e.g., 1536 for OpenAI ada-002, 384 for smaller models)

Driver support across all major SQL client libraries

Binary storage optimized for efficient I/O

Model Definitions — CREATE EXTERNAL MODEL



Declarative model definitions using CREATE EXTERNAL MODEL



Supported API types: Azure OpenAI, OpenAI, Ollama, ONNX Runtime



Points to a REST endpoint — models run outside the engine



System view for managing model definitions



Ground or cloud deployment

Embedding Generation

T-SQL functions to generate embeddings from your data

Text chunking: break large text into model-appropriate segments

Multi-modal embedding support

Works with any REST-accessible embedding model

Store embeddings in vector columns alongside your relational data

SQL Server 2025

Licensing, Features, and Cool DBA Stuff

Developer Edition Changes

Choose your target production deployment: Standard or Enterprise

Develop and test with no license costs

Features and limits now match your target deployment edition

Two flavors: **Developer Standard Edition** and Developer Enterprise Edition

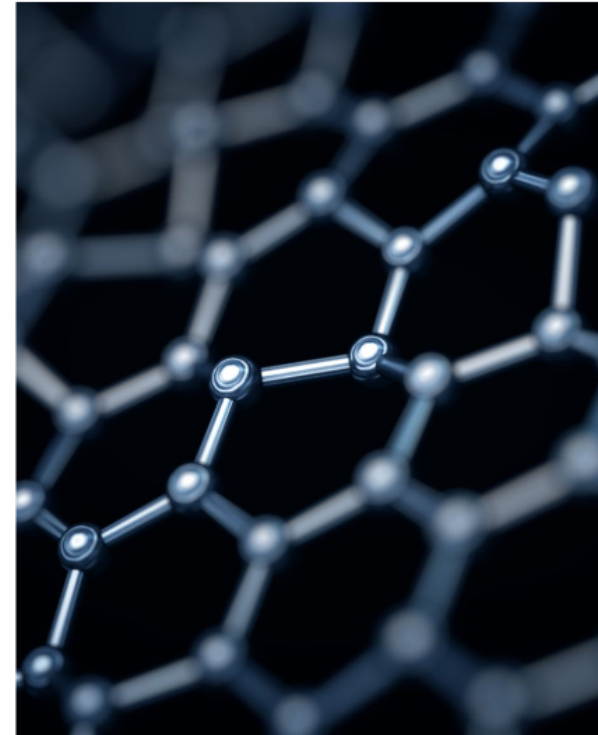
FAQ – Pricing, Licensing, and Editions

No pricing or licensing changes for SQL Server

Express: Now supports 50GB database, many new features included

Standard: Core limit increased from 24 → 32, memory from 128GB → 256GB, full Resource Governor

All new **features** are in both **Standard and Enterprise Editions**



Deprecated Features and Other Items



Discontinued: MDS, DQS, Synapse Link, Hot Add CPU, Purview Access Policies



PREVIEW_FEATURES database option: enables features in preview at GA



Power BI Report Server replaces SSRS for paid licenses

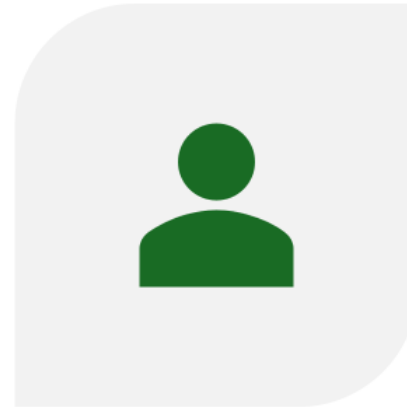


SSAS (Tabular) has performance improvements; SSIS still supported

Things You Need to Do



Use Query Store



Use ADR

Optimized Locking – The Problem



Traditional lock escalation: row > page > table



Causes blocking, deadlocks, and unpredictable concurrency



Applications forced to design around locking internals



Lock memory consumption grows with concurrent workloads

Optimized Locking – The Solution

Transaction ID (TID) locking
replaces traditional escalation

Hint free lock management built
into the engine

Already proven in Azure SQL
Database

No application code changes
required

Optimized Locking – Benefits



Dramatically improved concurrency for update-heavy workloads



Reduced lock memory overhead



Eliminated lock escalation blocking scenarios



Simpler troubleshooting — fewer locking-related issues

Intelligent Query Processing — What's New



CE (Cardinality Estimator) feedback for expressions



Optional parameter plans optimization (building on PSP from 2022)



DOP (Degree of Parallelism) feedback — now ON by default



Key theme: the optimizer learns and adapts, fewer code changes needed

Optimized *sp_executesql*

Better plan reuse for parameterized queries

Reduces compilation overhead

Significant impact for ORM-generated workloads (Entity Framework, Hibernate, etc.)

No application changes required

Query Store Enhancements

- Query Store on readable secondary replicas
- Persist statistics on secondaries
- Better insight into read workload performance on AG replicas
- Enhanced diagnostics for query performance regression



Tempdb Resource Governance

Control tempdb space consumption using Resource Governor workload groups

Prevent runaway queries from exhausting tempdb

Per-group tempdb space limits

Full Resource Governor now available in Standard Edition!

ABORT_QUERY_EXECUTION Hint

New query hint to forcefully terminate long-running queries after a threshold

Practical for BI/reporting environments with unpredictable query patterns

Complements Query Governor and Resource Governor

Other Performance Enhancements



ADR (Accelerated Database Recovery) in tempdb



tmpfs support for tempdb on Linux



Remove In-Memory OLTP from a database (finally!)



Persisted statistics for readable secondaries



Change tracking cleanup improvements

Backup Enhancements — ZSTD Compression

ZSTD backup compression algorithm

Significantly faster and better
compression ratios than legacy algorithm

Reduced backup windows and storage
costs

Easy to enable — just specify the new
compression option

Backup Enhancements — Azure and Secondary



Backups on secondary replicas
— offload backup workload
from primary



Backup to Azure immutable
storage — compliance and
ransomware protection



Backup to URL with Managed
Identity — passwordless
backup to Azure



Combines with Arc for
seamless configuration with
Azure Backup

One More Cool DBA Thing...

- Automatic Index Compaction
 - Announced through a blog post?!
- Maintains/optimizes indexes as a background process
 - Arguably reduces much of the need for index rebuild/reorg
- Reduces index bloat without direct DBA intervention

Thank you

Matt Gordon

mgordon@leantechniques.com

@sqlatspeed everywhere online

